



Week-10

Introduction to Database Systems

Shah Nawaz



Aggregate Functions, UNION, UNION ALL, INTERSECT

Relational Algebra

The header features a light blue background on the left with various school supplies: a pair of blue-handled scissors, a white eraser, a red pencil, a small blue sharpener, and a pair of compasses. On the right, a dark blue gradient background contains a faint, stylized image of a protractor and the text "Joining more than 2 tables" in white.

Joining more than 2 tables

Task:

"What are the employee ID, employee name, department name, and salary amount for each employee, considering the information from the employees, departments, and salaries tables, where employees are associated with their respective departments and salaries?"



Joining more than 2 tables

Here:

- The **employees** [employee_id, employee_name, department_id]
- The **departments** [department_id , department_name]
- The **salaries** [employee_id, salary_amount.

```
SELECT
    employees.employee_id, employees.employee_name, departments.department_name, salaries.salary_amount
FROM employees
JOIN
    departments ON employees.department_id = departments.department_id
JOIN
    salaries ON employees.employee_id = salaries.employee_id;
```

employee_id	employee_name	department_name	salary_amount
1	John Doe	Human Resources	60000.00
2	Jane Smith	IT Department	75000.00
3	Bob Johnson	Human Resources	55000.00



Joining more than 2 tables

Execution Detail

1.SELECT Clause:

- Specifies the columns to be included in the result set.
- It selects four columns: **employee_id**, **employee_name**, **department_name**, and **salary_amount**.

2.FROM Clause:

- Specifies the main table from which the data will be retrieved, which is the **employees** table in this case.

3.JOIN Clauses:

•First JOIN:

- Joins the **employees** table with the **departments** table based on the common column **department_id**.
- This allows the query to link employee information with their corresponding departments.
- Syntax: **JOIN departments ON employees.department_id = departments.department_id**

•Second JOIN:

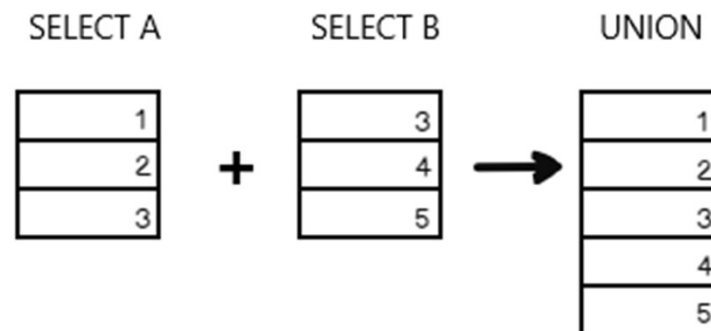
- Joins the result of the first join with the **salaries** table based on the common column **employee_id**.
- This links the employee information with their respective salaries.
- Syntax: **JOIN salaries ON employees.employee_id = salaries.employee_id**

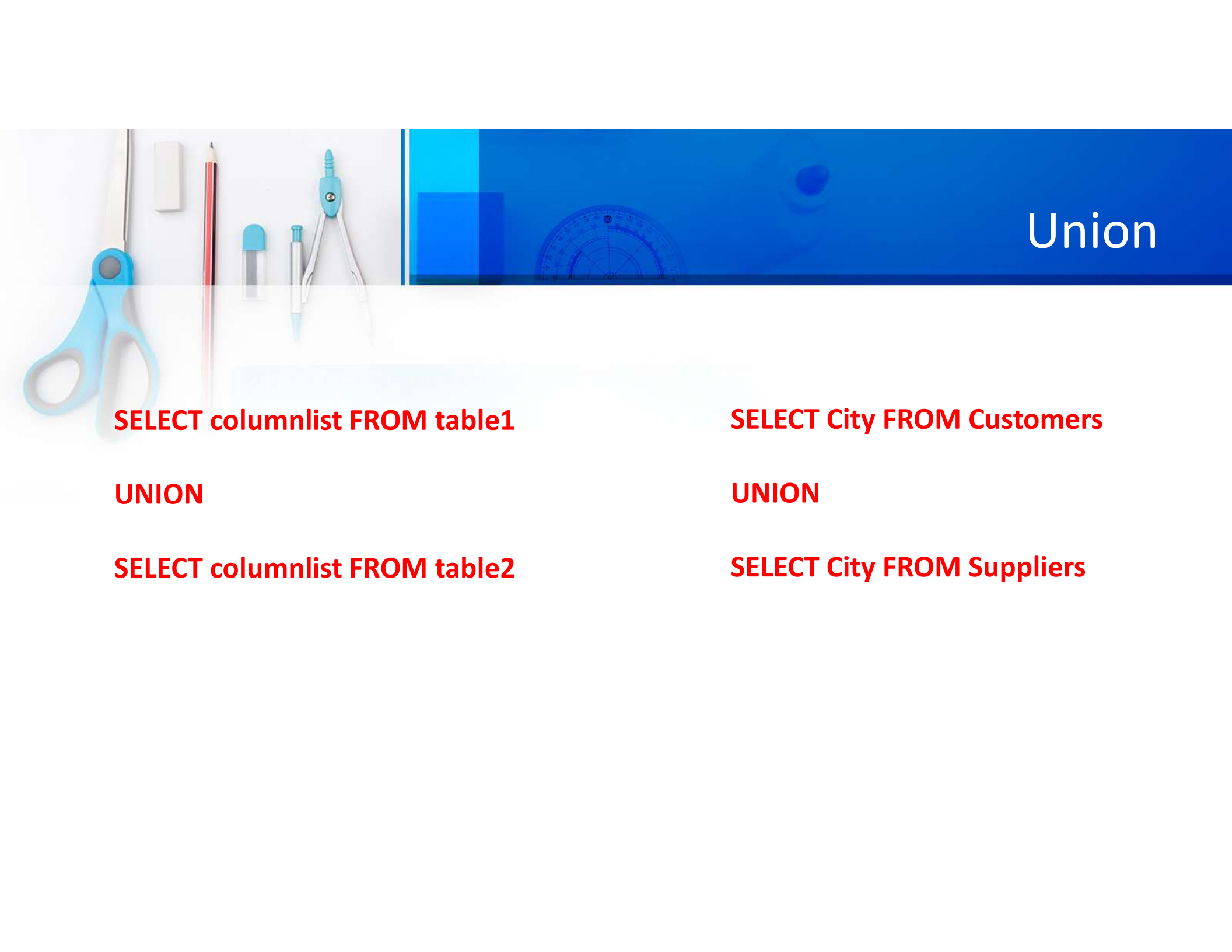
4.Result Set:

- The final result set includes columns from all three tables: **employee_id** and **employee_name** from the **employees** table, **department_name** from the **departments** table, and **salary_amount** from the **salaries** table.

Union

- The UNION operator is **used to combine** the result sets of two or more SELECT statements and returns a distinct set of rows.
- Duplicate rows are eliminated from the result set, so each row appears only once.
- The columns in the SELECT statements must be in the **same order**, have the **same data types**, and the **number of columns must be the same**.





Union

SELECT columnlist FROM table1

UNION

SELECT columnlist FROM table2

SELECT City FROM Customers

UNION

SELECT City FROM Suppliers



Union

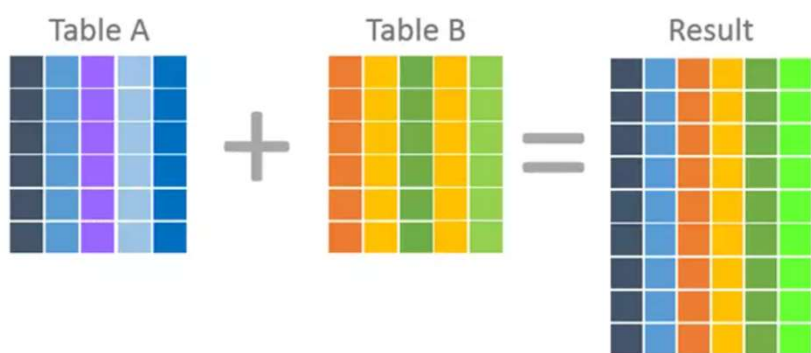
Name All people (Student + Teacher) From Lahore

```
SELECT CNIC, Name  
FROM Student  
where Address LIKE '%Lahore%'
```

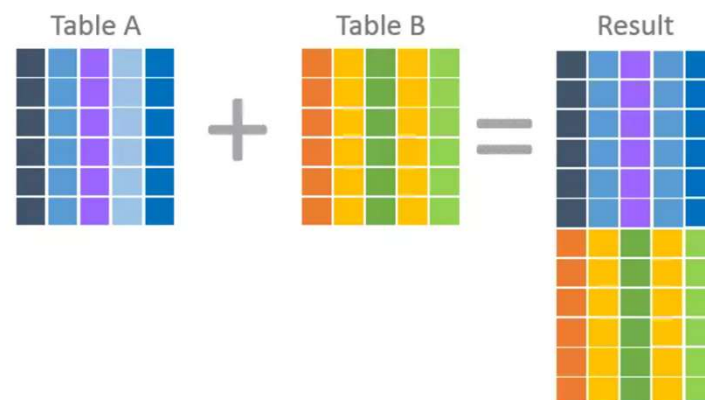
UNION

```
SELECT CNIC , Name  
FROM Teacher  
where Address LIKE '%Lahore%';
```

Union vs join



Join combines columns



Union joins rows



Union All

- The UNION ALL operator is similar to UNION but does not eliminate duplicate rows. It simply combines all the rows from the result sets, including duplicates.
- Because duplicate rows are not removed, UNION ALL is generally faster than UNION.
- The columns in the SELECT statements must be in the same order, have the same data types, and the number of columns must be the same.

SELECT columnlist FROM table1

UNION ALL

SELECT columnlist FROM table2

Union All

- The UNION operator is **used to combine** the result sets of two or more SELECT statements and returns a distinct set of rows.
- Duplicate rows are eliminated from the result set, so each row appears only once.
- The columns in the SELECT statements must be in the **same order**, have the **same data types**, and the **number of columns must be the same**.

SELECT City FROM Customers

UNION ALL

SELECT City FROM Suppliers





Union Compatibility

- The columns in the SELECT statements must be in the **same order**, have the **same data types**, and the **number of columns must be the same**.



Minus operation on sets

- To have the functionality of minus or Except we need to use NOT IN.

```
SELECT Age FROM employees  
WHERE employees.Age NOT IN  
(SELECT Age FROM supervisor);
```

The image features a horizontal header bar with a blue gradient. On the left side of the bar, there is a faint, light blue illustration of various geometric drawing tools: a pair of scissors, a pencil, an eraser, a sharpener, and a compass. The word "Intersection" is written in white, sans-serif font on the right side of the blue bar.

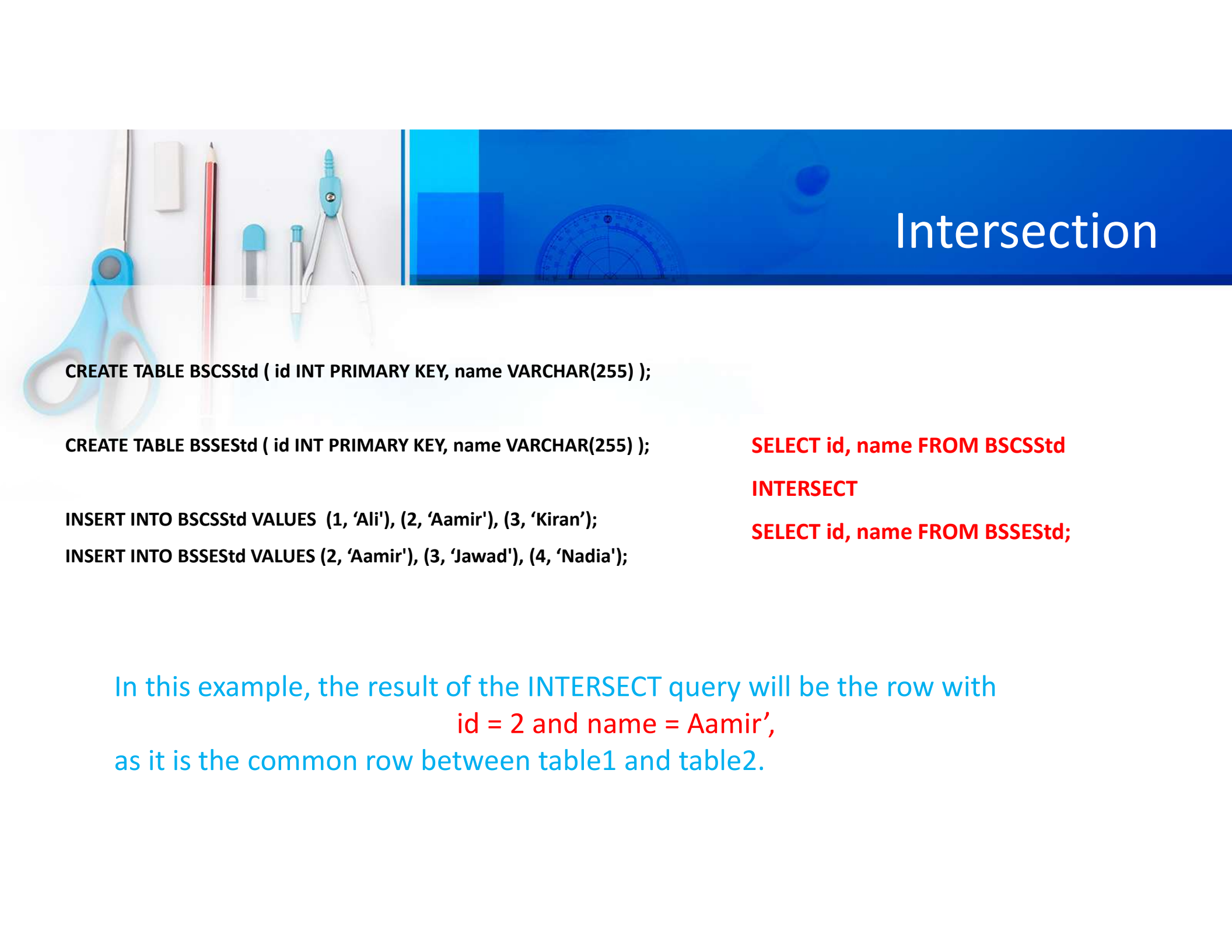
Intersection

The **INTERSECT** operator is used to combine the result sets of two SELECT statements, returning only the **common rows**.

```
SELECT column1, column2, ... FROM table1
```

```
INTERSECT
```

```
SELECT column1, column2, ... FROM table2;
```



Intersection

```
CREATE TABLE BSCSStd ( id INT PRIMARY KEY, name VARCHAR(255) );
```

```
CREATE TABLE BSSEStd ( id INT PRIMARY KEY, name VARCHAR(255) );
```

```
INSERT INTO BSCSStd VALUES (1, 'Ali'), (2, 'Aamir'), (3, 'Kiran');
```

```
INSERT INTO BSSEStd VALUES (2, 'Aamir'), (3, 'Jawad'), (4, 'Nadia');
```

```
SELECT id, name FROM BSCSStd
```

```
INTERSECT
```

```
SELECT id, name FROM BSSEStd;
```

In this example, the result of the INTERSECT query will be the row with
id = 2 and name = Aamir,
as it is the common row between table1 and table2.



Practice Questions

1. Show department names in which some workers works with salary > 30,000 or it is located in Stafford.
2. Show employee ssn's who are managers but don't have any dependents.
3. Show manager name who have no dependents.
4. Make a list of all project numbers for projects that involve an employee whose last name is smith either as a worker or as a manager of dept who controls the project.
5. List names of employees who do not work on any project.
6. Find the names of all employees who work on at least one project located in Houston but whose department has not located in Houston.



Relational Algebra



UNION

Let's consider a scenario where you want to retrieve the distinct values of "City" from both the "Customers" and "Suppliers" tables using the UNION operator. The SQL query and its relational algebra expression might look like this:

SELECT City FROM Customers UNION SELECT City FROM Suppliers;

In relational algebra, you might represent this as follows:

$\pi_{\text{City}}(\text{Customers}) \cup \pi_{\text{City}}(\text{Suppliers})$

Pronunciation: "Projection of the 'City' attribute from the 'Customers' table unioned with the projection of the 'City' attribute from the 'Suppliers' table."

Explanation:

- $\pi_{\text{City}}(\text{Customers})$ represents the Projection operation, selecting the "City" attribute from the "Customers" relation.
- $\pi_{\text{City}}(\text{Suppliers})$ represents the Projection operation, selecting the "City" attribute from the "Suppliers" relation.
- $\cup$ represents the Union operation, combining the distinct values of "City" from both projections.



UNION ALL

Let's consider a scenario where you want to retrieve all values of "City" from both the "Customers" and "Suppliers" tables, including duplicates, using the UNION ALL operator. The SQL query and its relational algebra expression might look like this:

SELECT City FROM Customers UNION ALL SELECT City FROM Suppliers;

In relational algebra, you might represent this as follows:

$\pi_{\text{City}}(\text{Customers}) \cup \text{ALL } \pi_{\text{City}}(\text{Suppliers})$

Pronunciation: "Projection of the 'City' attribute from the 'Customers' table union all with the projection of the 'City' attribute from the 'Suppliers' table."

Explanation:

- $\pi_{\text{City}}(\text{Customers})$ represents the Projection operation, selecting the "City" attribute from the "Customers" relation.
- $\pi_{\text{City}}(\text{Suppliers})$ represents the Projection operation, selecting the "City" attribute from the "Suppliers" relation.
- $\cup \text{ALL}$ represents the Union All operation, combining all values of "City" from both projections, including duplicates.



INTERSECT

Let's consider a scenario where you want to retrieve the common values of "City" from both the "Customers" and "Suppliers" tables using the INTERSECT operator. The SQL query and its relational algebra expression might look like this:

SELECT City FROM Customers INTERSECT SELECT City FROM Suppliers;

In relational algebra, you might represent this as follows:

$\pi_{\text{City}}(\text{Customers}) \cap \pi_{\text{City}}(\text{Suppliers})$

Pronunciation: "Projection of the 'City' attribute from the 'Customers' table intersect with the projection of the 'City' attribute from the 'Suppliers' table."

Explanation:

- $\pi_{\text{City}}(\text{Customers})$ represents the Projection operation, selecting the "City" attribute from the "Customers" relation.
- $\pi_{\text{City}}(\text{Suppliers})$ represents the Projection operation, selecting the "City" attribute from the "Suppliers" relation.
- $\cap$ represents the Intersection operation, retrieving the common values of "City" from both projections.



Practice Questions

Show department names in which some workers works with salary > 30,000 or it is located in Stafford.

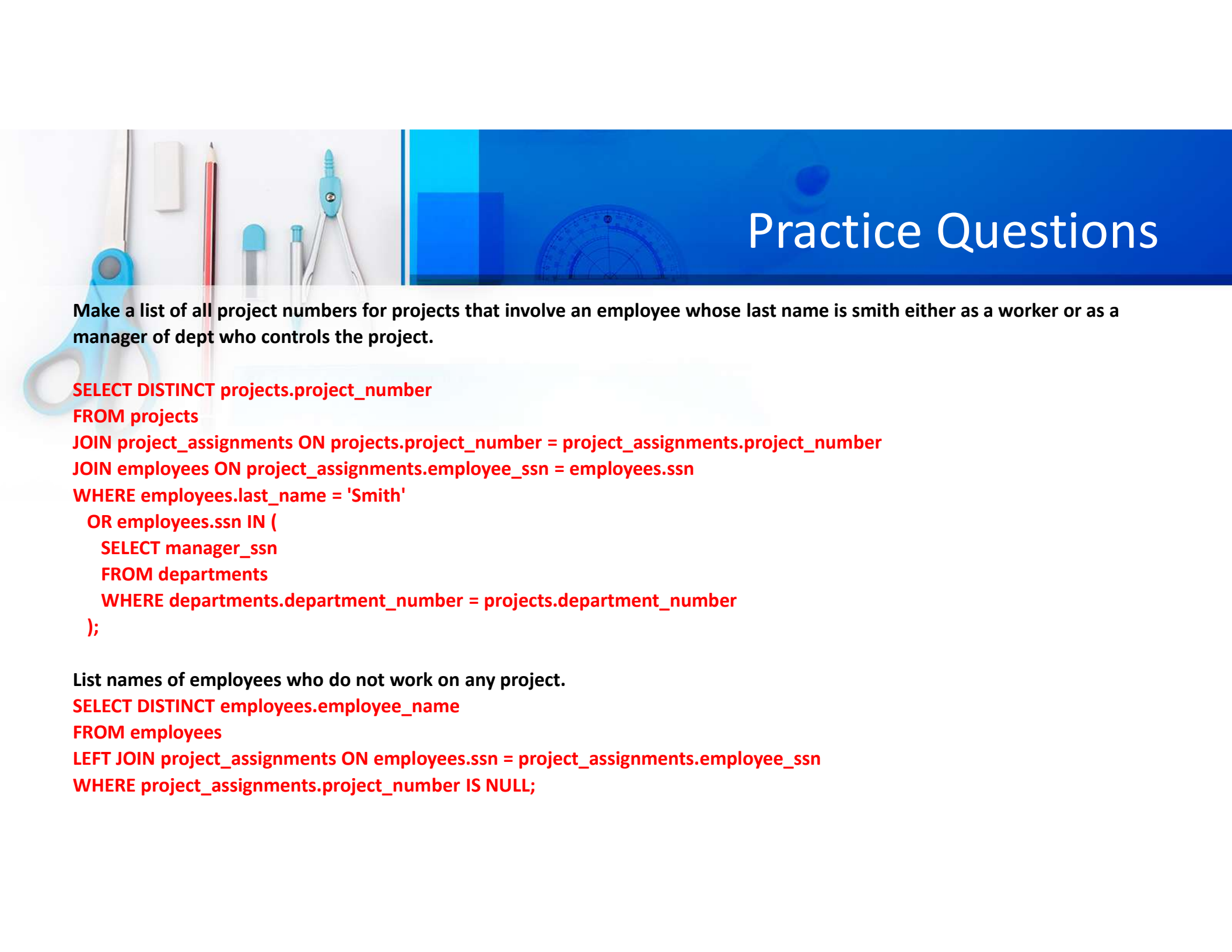
```
SELECT DISTINCT departments.department_name FROM employees JOIN departments ON employees.department_id =  
departments.department_id JOIN salaries ON employees.employee_id = salaries.employee_id WHERE salaries.salary_amount  
> 30000 OR departments.location = 'Stafford';
```

Show employee ssn's who are managers but don't have any dependents.

```
SELECT employees.ssn  
FROM employees  
LEFT JOIN dependents ON employees.ssn = dependents.employee_ssn  
WHERE employees.position = 'Manager' AND dependents.dependent_name IS NULL;
```

Show manager name who have no dependents.

```
SELECT DISTINCT employees.manager_name  
FROM employees  
LEFT JOIN dependents ON employees.ssn = dependents.employee_ssn  
WHERE employees.position = 'Manager' AND dependents.dependent_name IS NULL;
```



Practice Questions

Make a list of all project numbers for projects that involve an employee whose last name is smith either as a worker or as a manager of dept who controls the project.

```
SELECT DISTINCT projects.project_number
FROM projects
JOIN project_assignments ON projects.project_number = project_assignments.project_number
JOIN employees ON project_assignments.employee_ssn = employees.ssn
WHERE employees.last_name = 'Smith'
      OR employees.ssn IN (
        SELECT manager_ssn
        FROM departments
        WHERE departments.department_number = projects.department_number
      );
```

List names of employees who do not work on any project.

```
SELECT DISTINCT employees.employee_name
FROM employees
LEFT JOIN project_assignments ON employees.ssn = project_assignments.employee_ssn
WHERE project_assignments.project_number IS NULL;
```



Practice Questions

Fine the names of all employees who work on atleast one project located in Houston but whose department has not located in Houston.

```
SELECT DISTINCT employees.employee_name
FROM employees
JOIN project_assignments ON employees.ssn = project_assignments.employee_ssn
JOIN projects ON project_assignments.project_number = projects.project_number
JOIN departments ON employees.department_number = departments.department_number
WHERE projects.location = 'Houston' AND departments.location <> 'Houston';
```




Practice Questions

1. Show employee names who are managers and they have at least one dependent.
2. . Show employee name who are managers but they do not have dependents
3. Show employee name whose superssn is same as superssn of either “javed” or “ahmad”.
4. Show project name on which Aliya is not working.
5. Show all dependent name of employees working in IT department.
6. Show employee names who work in research department.
7. Show project names in which an employee ‘John Smith’ works.
8. Show all project names which are controlled by a department in which ‘John Smith’ works.
9. Show employee names who has a supervisor with the same salary as themselves.
10. Show employee names who do not work on any project.
11. Show department names which are not controlling any project and has a location in Houston



Thank You all!